

---

# Introduction to Data Analysis with Spark

This chapter provides a high-level overview of what Apache Spark is. If you are already familiar with Apache Spark and its components, feel free to jump ahead to [Chapter 2](#).

## What Is Apache Spark?

Apache Spark is a cluster computing platform designed to be *fast* and *general-purpose*.

On the speed side, Spark extends the popular MapReduce model to efficiently support more types of computations, including interactive queries and stream processing. Speed is important in processing large datasets, as it means the difference between exploring data interactively and waiting minutes or hours. One of the main features Spark offers for speed is the ability to run computations in memory, but the system is also more efficient than MapReduce for complex applications running on disk.

On the generality side, Spark is designed to cover a wide range of workloads that previously required separate distributed systems, including batch applications, iterative algorithms, interactive queries, and streaming. By supporting these workloads in the same engine, Spark makes it easy and inexpensive to *combine* different processing types, which is often necessary in production data analysis pipelines. In addition, it reduces the management burden of maintaining separate tools.

Spark is designed to be highly accessible, offering simple APIs in Python, Java, Scala, and SQL, and rich built-in libraries. It also integrates closely with other Big Data tools. In particular, Spark can run in Hadoop clusters and access any Hadoop data source, including Cassandra.

# A Unified Stack

The Spark project contains multiple closely integrated components. At its core, Spark is a “computational engine” that is responsible for scheduling, distributing, and monitoring applications consisting of many computational tasks across many worker machines, or a *computing cluster*. Because the core engine of Spark is both fast and general-purpose, it powers multiple higher-level components specialized for various workloads, such as SQL or machine learning. These components are designed to interoperate closely, letting you combine them like libraries in a software project.

A philosophy of tight integration has several benefits. First, all libraries and higher-level components in the stack benefit from improvements at the lower layers. For example, when Spark’s core engine adds an optimization, SQL and machine learning libraries automatically speed up as well. Second, the costs associated with running the stack are minimized, because instead of running 5–10 independent software systems, an organization needs to run only one. These costs include deployment, maintenance, testing, support, and others. This also means that each time a new component is added to the Spark stack, every organization that uses Spark will immediately be able to try this new component. This changes the cost of trying out a new type of data analysis from downloading, deploying, and learning a new software project to upgrading Spark.

Finally, one of the largest advantages of tight integration is the ability to build applications that seamlessly combine different processing models. For example, in Spark you can write one application that uses machine learning to classify data in real time as it is ingested from streaming sources. Simultaneously, analysts can query the resulting data, also in real time, via SQL (e.g., to join the data with unstructured log-files). In addition, more sophisticated data engineers and data scientists can access the same data via the Python shell for ad hoc analysis. Others might access the data in standalone batch applications. All the while, the IT team has to maintain only one system.

Here we will briefly introduce each of Spark’s components, shown in [Figure 1-1](#).

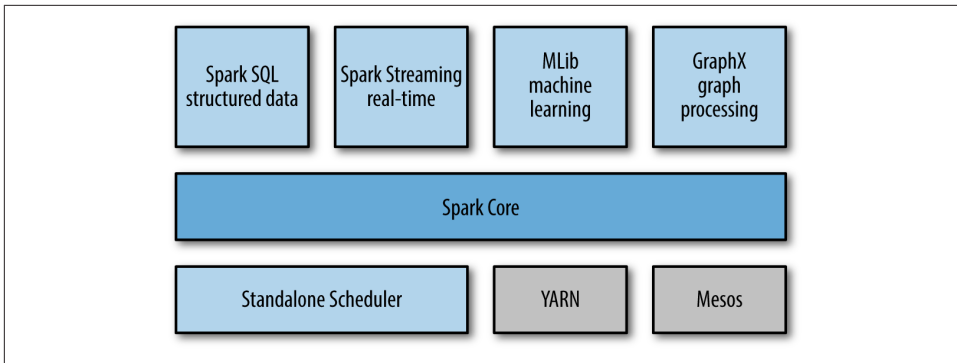


Figure 1-1. The Spark stack

## Spark Core

Spark Core contains the basic functionality of Spark, including components for task scheduling, memory management, fault recovery, interacting with storage systems, and more. Spark Core is also home to the API that defines *resilient distributed datasets* (RDDs), which are Spark’s main programming abstraction. RDDs represent a collection of items distributed across many compute nodes that can be manipulated in parallel. Spark Core provides many APIs for building and manipulating these collections.

## Spark SQL

Spark SQL is Spark’s package for working with structured data. It allows querying data via SQL as well as the Apache Hive variant of SQL—called the Hive Query Language (HQL)—and it supports many sources of data, including Hive tables, Parquet, and JSON. Beyond providing a SQL interface to Spark, Spark SQL allows developers to intermix SQL queries with the programmatic data manipulations supported by RDDs in Python, Java, and Scala, all within a single application, thus combining SQL with complex analytics. This tight integration with the rich computing environment provided by Spark makes Spark SQL unlike any other open source data warehouse tool. Spark SQL was added to Spark in version 1.0.

Shark was an older SQL-on-Spark project out of the University of California, Berkeley, that modified Apache Hive to run on Spark. It has now been replaced by Spark SQL to provide better integration with the Spark engine and language APIs.

## Spark Streaming

Spark Streaming is a Spark component that enables processing of live streams of data. Examples of data streams include logfiles generated by production web servers, or queues of messages containing status updates posted by users of a web service. Spark

Streaming provides an API for manipulating data streams that closely matches the Spark Core’s RDD API, making it easy for programmers to learn the project and move between applications that manipulate data stored in memory, on disk, or arriving in real time. Underneath its API, Spark Streaming was designed to provide the same degree of fault tolerance, throughput, and scalability as Spark Core.

## MLlib

Spark comes with a library containing common machine learning (ML) functionality, called MLlib. MLlib provides multiple types of machine learning algorithms, including classification, regression, clustering, and collaborative filtering, as well as supporting functionality such as model evaluation and data import. It also provides some lower-level ML primitives, including a generic gradient descent optimization algorithm. All of these methods are designed to scale out across a cluster.

## GraphX

GraphX is a library for manipulating graphs (e.g., a social network’s friend graph) and performing graph-parallel computations. Like Spark Streaming and Spark SQL, GraphX extends the Spark RDD API, allowing us to create a directed graph with arbitrary properties attached to each vertex and edge. GraphX also provides various operators for manipulating graphs (e.g., subgraph and mapVertices) and a library of common graph algorithms (e.g., PageRank and triangle counting).

## Cluster Managers

Under the hood, Spark is designed to efficiently scale up from one to many thousands of compute nodes. To achieve this while maximizing flexibility, Spark can run over a variety of *cluster managers*, including Hadoop YARN, Apache Mesos, and a simple cluster manager included in Spark itself called the Standalone Scheduler. If you are just installing Spark on an empty set of machines, the Standalone Scheduler provides an easy way to get started; if you already have a Hadoop YARN or Mesos cluster, however, Spark’s support for these cluster managers allows your applications to also run on them. [Chapter 7](#) explores the different options and how to choose the correct cluster manager.

## Who Uses Spark, and for What?

Because Spark is a general-purpose framework for cluster computing, it is used for a diverse range of applications. In the [Preface](#) we outlined two groups of readers that this book targets: data scientists and engineers. Let’s take a closer look at each group and how it uses Spark. Unsurprisingly, the typical use cases differ between the two,

but we can roughly classify them into two categories, *data science* and *data applications*.

Of course, these are imprecise disciplines and usage patterns, and many folks have skills from both, sometimes playing the role of the investigating data scientist, and then “changing hats” and writing a hardened data processing application. Nonetheless, it can be illuminating to consider the two groups and their respective use cases separately.

## Data Science Tasks

Data science, a discipline that has been emerging over the past few years, centers on analyzing data. While there is no standard definition, for our purposes a *data scientist* is somebody whose main task is to analyze and model data. Data scientists may have experience with SQL, statistics, predictive modeling (machine learning), and programming, usually in Python, Matlab, or R. Data scientists also have experience with techniques necessary to transform data into formats that can be analyzed for insights (sometimes referred to as *data wrangling*).

Data scientists use their skills to analyze data with the goal of answering a question or discovering insights. Oftentimes, their workflow involves ad hoc analysis, so they use interactive shells (versus building complex applications) that let them see results of queries and snippets of code in the least amount of time. Spark’s speed and simple APIs shine for this purpose, and its built-in libraries mean that many algorithms are available out of the box.

Spark supports the different tasks of data science with a number of components. The Spark shell makes it easy to do interactive data analysis using Python or Scala. Spark SQL also has a separate SQL shell that can be used to do data exploration using SQL, or Spark SQL can be used as part of a regular Spark program or in the Spark shell. Machine learning and data analysis is supported through the MLlib libraries. In addition, there is support for calling out to external programs in Matlab or R. Spark enables data scientists to tackle problems with larger data sizes than they could before with tools like R or Pandas.

Sometimes, after the initial exploration phase, the work of a data scientist will be “productized,” or extended, hardened (i.e., made fault-tolerant), and tuned to become a production data processing application, which itself is a component of a business application. For example, the initial investigation of a data scientist might lead to the creation of a production recommender system that is integrated into a web application and used to generate product suggestions to users. Often it is a different person or team that leads the process of productizing the work of the data scientists, and that person is often an engineer.

## Data Processing Applications

The other main use case of Spark can be described in the context of the engineer persona. For our purposes here, we think of engineers as a large class of software developers who use Spark to build production data processing applications. These developers usually have an understanding of the principles of software engineering, such as encapsulation, interface design, and object-oriented programming. They frequently have a degree in computer science. They use their engineering skills to design and build software systems that implement a business use case.

For engineers, Spark provides a simple way to parallelize these applications across clusters, and hides the complexity of distributed systems programming, network communication, and fault tolerance. The system gives them enough control to monitor, inspect, and tune applications while allowing them to implement common tasks quickly. The modular nature of the API (based on passing distributed collections of objects) makes it easy to factor work into reusable libraries and test it locally.

Spark's users choose to use it for their data processing applications because it provides a wide variety of functionality, is easy to learn and use, and is mature and reliable.

## A Brief History of Spark

Spark is an open source project that has been built and is maintained by a thriving and diverse community of developers. If you or your organization are trying Spark for the first time, you might be interested in the history of the project. Spark started in 2009 as a research project in the UC Berkeley RAD Lab, later to become the AMPLab. The researchers in the lab had previously been working on Hadoop MapReduce, and observed that MapReduce was inefficient for iterative and interactive computing jobs. Thus, from the beginning, Spark was designed to be fast for interactive queries and iterative algorithms, bringing in ideas like support for in-memory storage and efficient fault recovery.

Research papers were published about Spark at academic conferences and soon after its creation in 2009, it was already 10–20× faster than MapReduce for certain jobs.

Some of Spark's first users were other groups inside UC Berkeley, including machine learning researchers such as the Mobile Millennium project, which used Spark to monitor and predict traffic congestion in the San Francisco Bay Area. In a very short time, however, many external organizations began using Spark, and today, over 50 organizations list themselves on the [Spark PoweredBy page](#), and dozens speak about their use cases at Spark community events such as [Spark Meetups](#) and the [Spark Summit](#). In addition to UC Berkeley, major contributors to Spark include Databricks, Yahoo!, and Intel.

In 2011, the AMPLab started to develop higher-level components on Spark, such as Shark (Hive on Spark)<sup>1</sup> and Spark Streaming. These and other components are sometimes referred to as the **Berkeley Data Analytics Stack (BDAS)**.

Spark was first open sourced in March 2010, and was transferred to the Apache Software Foundation in June 2013, where it is now a top-level project.

## Spark Versions and Releases

Since its creation, Spark has been a very active project and community, with the number of contributors growing with each release. Spark 1.0 had over 100 individual contributors. Though the level of activity has rapidly grown, the community continues to release updated versions of Spark on a regular schedule. Spark 1.0 was released in May 2014. This book focuses primarily on Spark 1.1.0 and beyond, though most of the concepts and examples also work in earlier versions.

## Storage Layers for Spark

Spark can create distributed datasets from any file stored in the Hadoop distributed filesystem (HDFS) or other storage systems supported by the Hadoop APIs (including your local filesystem, Amazon S3, Cassandra, Hive, HBase, etc.). It's important to remember that Spark does not require Hadoop; it simply has support for storage systems implementing the Hadoop APIs. Spark supports text files, SequenceFiles, Avro, Parquet, and any other Hadoop InputFormat. We will look at interacting with these data sources in **Chapter 5**.

---

<sup>1</sup> Shark has been replaced by Spark SQL.



---

# Downloading Spark and Getting Started

In this chapter we will walk through the process of downloading and running Spark in local mode on a single computer. This chapter was written for anybody who is new to Spark, including both data scientists and engineers.

Spark can be used from Python, Java, or Scala. To benefit from this book, you don't need to be an expert programmer, but we do assume that you are comfortable with the basic syntax of at least one of these languages. We will include examples in all languages wherever possible.

Spark itself is written in Scala, and runs on the Java Virtual Machine (JVM). To run Spark on either your laptop or a cluster, all you need is an installation of Java 6 or newer. If you wish to use the Python API you will also need a Python interpreter (version 2.6 or newer). Spark does not yet work with Python 3.

## Downloading Spark

The first step to using Spark is to download and unpack it. Let's start by downloading a recent precompiled released version of Spark. Visit <http://spark.apache.org/downloads.html>, select the package type of “Pre-built for Hadoop 2.4 and later,” and click “Direct Download.” This will download a compressed TAR file, or *tarball*, called *spark-1.2.0-bin-hadoop2.4.tgz*.



Windows users may run into issues installing Spark into a directory with a space in the name. Instead, install Spark in a directory with no space (e.g., `C:\spark`).

You don't need to have Hadoop, but if you have an existing Hadoop cluster or HDFS installation, download the matching version. You can do so from <http://spark.apache.org/downloads.html> by selecting a different package type, but they will have slightly different filenames. Building from source is also possible; you can find the latest source code on [GitHub](#) or select the package type of "Source Code" when downloading.



Most Unix and Linux variants, including Mac OS X, come with a command-line tool called `tar` that can be used to unpack TAR files. If your operating system does not have the `tar` command installed, try searching the Internet for a free TAR extractor—for example, on Windows, you may wish to try 7-Zip.

Now that we have downloaded Spark, let's unpack it and take a look at what comes with the default Spark distribution. To do that, open a terminal, change to the directory where you downloaded Spark, and `untar` the file. This will create a new directory with the same name but without the final `.tgz` suffix. Change into that directory and see what's inside. You can use the following commands to accomplish all of that:

```
cd ~
tar -xf spark-1.2.0-bin-hadoop2.4.tgz
cd spark-1.2.0-bin-hadoop2.4
ls
```

In the line containing the `tar` command, the `x` flag tells `tar` we are extracting files, and the `f` flag specifies the name of the tarball. The `ls` command lists the contents of the Spark directory. Let's briefly consider the names and purposes of some of the more important files and directories you see here that come with Spark:

*README.md*

Contains short instructions for getting started with Spark.

*bin*

Contains executable files that can be used to interact with Spark in various ways (e.g., the Spark shell, which we will cover later in this chapter).

*core, streaming, python, ...*

Contains the source code of major components of the Spark project.

*examples*

Contains some helpful Spark standalone jobs that you can look at and run to learn about the Spark API.

Don't worry about the large number of directories and files the Spark project comes with; we will cover most of these in the rest of this book. For now, let's dive right in and try out Spark's Python and Scala shells. We will start by running some of the

examples that come with Spark. Then we will write, compile, and run a simple Spark job of our own.

All of the work we will do in this chapter will be with Spark running in *local mode*; that is, nondistributed mode, which uses only a single machine. Spark can run in a variety of different modes, or environments. Beyond local mode, Spark can also be run on Mesos, YARN, or the Standalone Scheduler included in the Spark distribution. We will cover the various deployment modes in detail in [Chapter 7](#).

## Introduction to Spark's Python and Scala Shells

Spark comes with interactive shells that enable ad hoc data analysis. Spark's shells will feel familiar if you have used other shells such as those in R, Python, and Scala, or operating system shells like Bash or the Windows command prompt.

Unlike most other shells, however, which let you manipulate data using the disk and memory on a single machine, Spark's shells allow you to interact with data that is distributed on disk or in memory across many machines, and Spark takes care of automatically distributing this processing.

Because Spark can load data into memory on the worker nodes, many distributed computations, even ones that process terabytes of data across dozens of machines, can run in a few seconds. This makes the sort of iterative, ad hoc, and exploratory analysis commonly done in shells a good fit for Spark. Spark provides both Python and Scala shells that have been augmented to support connecting to a cluster.



Most of this book includes code in all of Spark's languages, but interactive shells are available only in Python and Scala. Because a shell is very useful for learning the API, we recommend using one of these languages for these examples even if you are a Java developer. The API is similar in every language.

The easiest way to demonstrate the power of Spark's shells is to start using one of them for some simple data analysis. Let's walk through the example from the [Quick Start Guide](#) in the official Spark documentation.

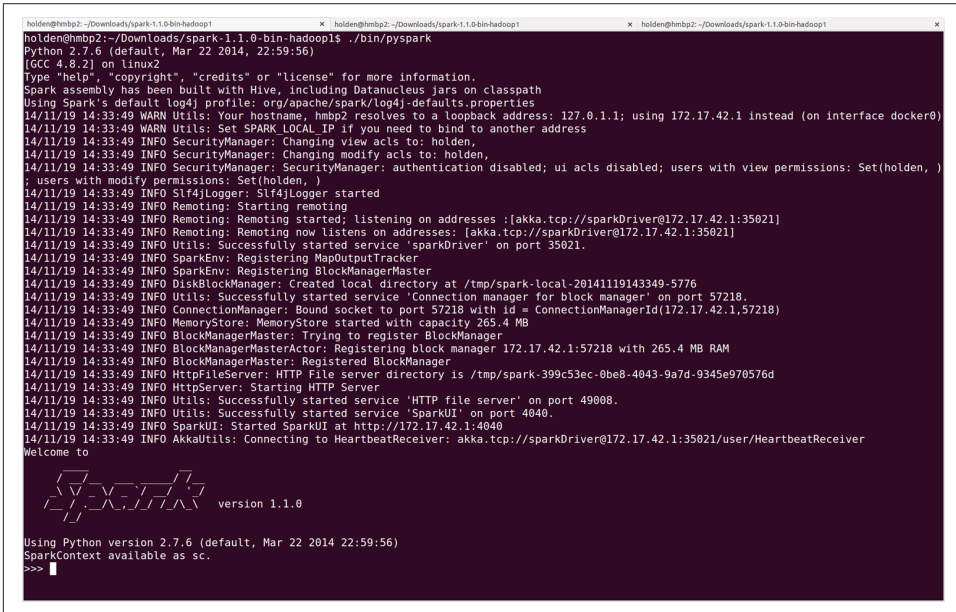
The first step is to open up one of Spark's shells. To open the Python version of the Spark shell, which we also refer to as the PySpark Shell, go into your Spark directory and type:

```
bin/pyspark
```

(Or `bin\pyspark` in Windows.) To open the Scala version of the shell, type:

```
bin/spark-shell
```

The shell prompt should appear within a few seconds. When the shell starts, you will notice a lot of log messages. You may need to press Enter once to clear the log output and get to a shell prompt. **Figure 2-1** shows what the PySpark shell looks like when you open it.

The image shows a terminal window titled 'holden@hmbp2: ~/Downloads/spark-1.1.0-bin-hadoop1'. The terminal displays the output of running 'pyspark'. It starts with a Python version notice (2.7.6), a GCC version notice (4.8.2), and a 'Type "help"' prompt. Then, it shows a series of INFO and WARN messages from the Spark logging system, including details about the hostname, security manager, and various services starting up. The messages end with 'Welcome to' and the PySpark logo. Below the logo, it says 'version 1.1.0' and 'Using Python version 2.7.6 (default, Mar 22 2014 22:59:56)'. The prompt 'SparkContext available as sc.' is shown, followed by a '++>' prompt.

*Figure 2-1. The PySpark shell with default logging output*

You may find the logging statements that get printed in the shell distracting. You can control the verbosity of the logging. To do this, you can create a file in the *conf* directory called *log4j.properties*. The Spark developers already include a template for this file called *log4j.properties.template*. To make the logging less verbose, make a copy of *conf/log4j.properties.template* called *conf/log4j.properties* and find the following line:

```
log4j.rootCategory=INFO, console
```

Then lower the log level so that we show only the WARN messages, and above by changing it to the following:

```
log4j.rootCategory=WARN, console
```

When you reopen the shell, you should see less output (**Figure 2-2**).

```
holden@hmbp2: ~/Downloads/spark-1.1.0-bin-hadoop1
holden@hmbp2:~/Downloads/spark-1.1.0-bin-hadoop1$ ./bin/pyspark
Python 2.7.6 (default, Mar 22 2014, 22:59:56)
[GCC 4.8.2] on linux2
Type "help", "copyright", "credits" or "license" for more information.
Spark assembly has been built with Hive, including Datanucleus jars on classpath
14/11/19 14:38:03 WARN Utils: Your hostname, hmbp2 resolves to a loopback address: 127.0.1.1; using 172.17.42.1 instead (on interface docker0)
14/11/19 14:38:03 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Welcome to

  ____      __
  /  _ \    / /
 /  /_\  \  / /
/_  _  _\/_/ /_
/_  _  _\/_/ /_
/_  _  _\/_/ /_
/_  _  _\/_/ /_

version 1.1.0

Using Python version 2.7.6 (default, Mar 22 2014 22:59:56)
SparkContext available as sc.
>>>
```

Figure 2-2. The PySpark shell with less logging output



## Using IPython

IPython is an enhanced Python shell that many Python users prefer, offering features such as tab completion. You can find instructions for installing it at <http://ipython.org>. You can use IPython with Spark by setting the IPYTHON environment variable to 1:

```
IPYTHON=1 ./bin/pyspark
```

To use the IPython Notebook, which is a web-browser-based version of IPython, use:

```
IPYTHON_OPTS="notebook" ./bin/pyspark
```

On Windows, set the variable and run the shell as follows:

```
set IPYTHON=1
bin\pyspark
```

In Spark, we express our computation through operations on distributed collections that are automatically parallelized across the cluster. These collections are called *resilient distributed datasets*, or RDDs. RDDs are Spark's fundamental abstraction for distributed data and computation.

Before we say more about RDDs, let's create one in the shell from a local text file and do some very simple ad hoc analysis by following [Example 2-1](#) for Python or [Example 2-2](#) for Scala.

### Example 2-1. Python line count

```
>>> lines = sc.textFile("README.md") # Create an RDD called lines

>>> lines.count() # Count the number of items in this RDD
127
>>> lines.first() # First item in this RDD, i.e. first line of README.md
u'# Apache Spark'
```

### Example 2-2. Scala line count

```
scala> val lines = sc.textFile("README.md") // Create an RDD called lines
lines: spark.RDD[String] = MappedRDD[...]

scala> lines.count() // Count the number of items in this RDD
res0: Long = 127

scala> lines.first() // First item in this RDD, i.e. first line of README.md
res1: String = # Apache Spark
```

To exit either shell, press Ctrl-D.



We will discuss it more in [Chapter 7](#), but one of the messages you may have noticed is `INFO SparkUI: Started SparkUI at http://[ipaddress]:4040`. You can access the Spark UI there and see all sorts of information about your tasks and cluster.

In Examples 2-1 and 2-2, the variable called `lines` is an RDD, created here from a text file on our local machine. We can run various parallel operations on the RDD, such as counting the number of elements in the dataset (here, lines of text in the file) or printing the first one. We will discuss RDDs in great depth in later chapters, but before we go any further, let's take a moment now to introduce basic Spark concepts.

## Introduction to Core Spark Concepts

Now that you have run your first Spark code using the shell, it's time to learn about programming in it in more detail.

At a high level, every Spark application consists of a *driver program* that launches various parallel operations on a cluster. The driver program contains your application's main function and defines distributed datasets on the cluster, then applies operations to them. In the preceding examples, the driver program was the Spark shell itself, and you could just type in the operations you wanted to run.

Driver programs access Spark through a `SparkContext` object, which represents a connection to a computing cluster. In the shell, a `SparkContext` is automatically

created for you as the variable called `sc`. Try printing out `sc` to see its type, as shown in [Example 2-3](#).

#### Example 2-3. Examining the `sc` variable

```
>>> sc
<pyspark.context.SparkContext object at 0x1025b8f90>
```

Once you have a `SparkContext`, you can use it to build RDDs. In [Examples 2-1](#) and [2-2](#), we called `sc.textFile()` to create an RDD representing the lines of text in a file. We can then run various operations on these lines, such as `count()`.

To run these operations, driver programs typically manage a number of nodes called *executors*. For example, if we were running the `count()` operation on a cluster, different machines might count lines in different ranges of the file. Because we just ran the Spark shell locally, it executed all its work on a single machine—but you can connect the same shell to a cluster to analyze data in parallel. [Figure 2-3](#) shows how Spark executes on a cluster.

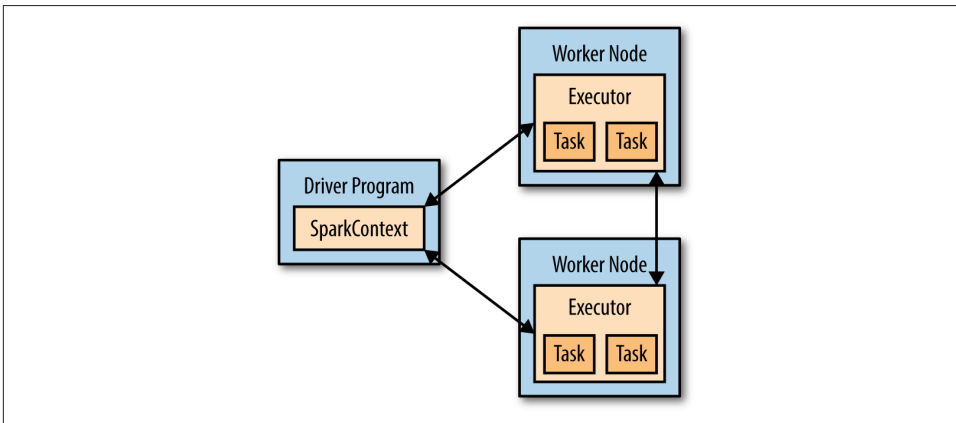


Figure 2-3. Components for distributed execution in Spark

Finally, a lot of Spark’s API revolves around passing functions to its operators to run them on the cluster. For example, we could extend our `README` example by *filtering* the lines in the file that contain a word, such as *Python*, as shown in [Example 2-4](#) (for Python) and [Example 2-5](#) (for Scala).

#### Example 2-4. Python filtering example

```
>>> lines = sc.textFile("README.md")
>>> pythonLines = lines.filter(lambda line: "Python" in line)
```

```
>>> pythonLines.first()
u'## Interactive Python Shell'
```

*Example 2-5. Scala filtering example*

```
scala> val lines = sc.textFile("README.md") // Create an RDD called lines
lines: spark.RDD[String] = MappedRDD[...]

scala> val pythonLines = lines.filter(line => line.contains("Python"))
pythonLines: spark.RDD[String] = FilteredRDD[...]

scala> pythonLines.first()
res0: String = ## Interactive Python Shell
```

## Passing Functions to Spark

If you are unfamiliar with the `lambda` or `=>` syntax in Examples 2-4 and 2-5, it is a shorthand way to define functions inline in Python and Scala. When using Spark in these languages, you can also define a function separately and then pass its name to Spark. For example, in Python:

```
def hasPython(line):
    return "Python" in line

pythonLines = lines.filter(hasPython)
```

Passing functions to Spark is also possible in Java, but in this case they are defined as classes, implementing an interface called `Function`. For example:

```
JavaRDD<String> pythonLines = lines.filter(
    new Function<String, Boolean>() {
        Boolean call(String line) { return line.contains("Python"); }
    }
);
```

Java 8 introduces shorthand syntax called *lambdas* that looks similar to Python and Scala. Here is how the code would look with this syntax:

```
JavaRDD<String> pythonLines = lines.filter(line -> line.contains("Python"));
```

We discuss passing functions further in “[Passing Functions to Spark](#)” on page 30.

While we will cover the Spark API in more detail later, a lot of its magic is that function-based operations like `filter` *also* parallelize across the cluster. That is, Spark automatically takes your function (e.g., `line.contains("Python")`) and ships it to executor nodes. Thus, you can write code in a single driver program and automatically have parts of it run on multiple nodes. [Chapter 3](#) covers the RDD API in detail.

# Standalone Applications

The final piece missing in this quick tour of Spark is how to use it in standalone programs. Apart from running interactively, Spark can be linked into standalone applications in either Java, Scala, or Python. The main difference from using it in the shell is that you need to initialize your own `SparkContext`. After that, the API is the same.

The process of linking to Spark varies by language. In Java and Scala, you give your application a Maven dependency on the `spark-core` artifact. As of the time of writing, the latest Spark version is 1.2.0, and the Maven coordinates for that are:

```
groupId = org.apache.spark
artifactId = spark-core_2.10
version = 1.2.0
```

Maven is a popular package management tool for Java-based languages that lets you link to libraries in public repositories. You can use Maven itself to build your project, or use other tools that can talk to the Maven repositories, including Scala's `sbt` tool or Gradle. Popular integrated development environments like Eclipse also allow you to directly add a Maven dependency to a project.

In Python, you simply write applications as Python scripts, but you must run them using the `bin/spark-submit` script included in Spark. The `spark-submit` script includes the Spark dependencies for us in Python. This script sets up the environment for Spark's Python API to function. Simply run your script with the line given in [Example 2-6](#).

*Example 2-6. Running a Python script*

```
bin/spark-submit my_script.py
```

(Note that you will have to use backslashes instead of forward slashes on Windows.)

## Initializing a SparkContext

Once you have linked an application to Spark, you need to import the Spark packages in your program and create a `SparkContext`. You do so by first creating a `SparkConf` object to configure your application, and then building a `SparkContext` for it. [Examples 2-7](#) through [2-9](#) demonstrate this in each supported language.

*Example 2-7. Initializing Spark in Python*

```
from pyspark import SparkConf, SparkContext

conf = SparkConf().setMaster("local").setAppName("My App")
sc = SparkContext(conf = conf)
```

### Example 2-8. Initializing Spark in Scala

```
import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._

val conf = new SparkConf().setMaster("local").setAppName("My App")
val sc = new SparkContext(conf)
```

### Example 2-9. Initializing Spark in Java

```
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaSparkContext;

SparkConf conf = new SparkConf().setMaster("local").setAppName("My App");
JavaSparkContext sc = new JavaSparkContext(conf);
```

These examples show the minimal way to initialize a SparkContext, where you pass two parameters:

- A *cluster URL*, namely `local` in these examples, which tells Spark how to connect to a cluster. `local` is a special value that runs Spark on one thread on the local machine, without connecting to a cluster.
- An *application name*, namely `My App` in these examples. This will identify your application on the cluster manager's UI if you connect to a cluster.

Additional parameters exist for configuring how your application executes or adding code to be shipped to the cluster, but we will cover these in later chapters of the book.

After you have initialized a SparkContext, you can use all the methods we showed before to create RDDs (e.g., from a text file) and manipulate them.

Finally, to shut down Spark, you can either call the `stop()` method on your SparkContext, or simply exit the application (e.g., with `System.exit(0)` or `sys.exit()`).

This quick overview should be enough to let you run a standalone Spark application on your laptop. For more advanced configuration, [Chapter 7](#) will cover how to connect your application to a cluster, including packaging your application so that its code is automatically shipped to worker nodes. For now, please refer to the [Quick Start Guide](#) in the official Spark documentation.

## Building Standalone Applications

This wouldn't be a complete introductory chapter of a Big Data book if we didn't have a word count example. On a single machine, implementing word count is simple, but in distributed frameworks it is a common example because it involves reading and combining data from many worker nodes. We will look at building and

packaging a simple word count example with both sbt and Maven. All of our examples can be built together, but to illustrate a stripped-down build with minimal dependencies we have a separate smaller project underneath the *learning-spark-examples/mini-complete-example* directory, as you can see in Examples 2-10 (Java) and 2-11 (Scala).

*Example 2-10. Word count Java application—don't worry about the details yet*

```
// Create a Java Spark Context
SparkConf conf = new SparkConf().setAppName("wordCount");
JavaSparkContext sc = new JavaSparkContext(conf);
// Load our input data.
JavaRDD<String> input = sc.textFile(inputFile);
// Split up into words.
JavaRDD<String> words = input.flatMap(
    new FlatMapFunction<String, String>() {
        public Iterable<String> call(String x) {
            return Arrays.asList(x.split(" "));
        }
    });
// Transform into pairs and count.
JavaPairRDD<String, Integer> counts = words.mapToPair(
    new PairFunction<String, String, Integer>(){
        public Tuple2<String, Integer> call(String x){
            return new Tuple2(x, 1);
        }
    }).reduceByKey(new Function2<Integer, Integer, Integer>(){
        public Integer call(Integer x, Integer y){ return x + y;}});
// Save the word count back out to a text file, causing evaluation.
counts.saveAsTextFile(outputFile);
```

*Example 2-11. Word count Scala application—don't worry about the details yet*

```
// Create a Scala Spark Context.
val conf = new SparkConf().setAppName("wordCount")
val sc = new SparkContext(conf)
// Load our input data.
val input = sc.textFile(inputFile)
// Split it up into words.
val words = input.flatMap(line => line.split(" "))
// Transform into pairs and count.
val counts = words.map(word => (word, 1)).reduceByKey{case (x, y) => x + y}
// Save the word count back out to a text file, causing evaluation.
counts.saveAsTextFile(outputFile)
```

We can build these applications using very simple build files with both sbt (Example 2-12) and Maven (Example 2-13). We've marked the Spark Core dependency as provided so that, later on, when we use an assembly JAR we don't include the spark-core JAR, which is already on the classpath of the workers.

### Example 2-12. sbt build file

```
name := "learning-spark-mini-example"

version := "0.0.1"

scalaVersion := "2.10.4"

// additional libraries
libraryDependencies += Seq(
  "org.apache.spark" %% "spark-core" % "1.2.0" % "provided"
)
```

### Example 2-13. Maven build file

```
<project>
  <groupId>com.oreilly.learningsparkexamples.mini</groupId>
  <artifactId>learning-spark-mini-example</artifactId>
  <modelVersion>4.0.0</modelVersion>
  <name>example</name>
  <packaging>jar</packaging>
  <version>0.0.1</version>
  <dependencies>
    <dependency> <!-- Spark dependency -->
      <groupId>org.apache.spark</groupId>
      <artifactId>spark-core_2.10</artifactId>
      <version>1.2.0</version>
      <scope>provided</scope>
    </dependency>
  </dependencies>
  <properties>
    <java.version>1.6</java.version>
  </properties>
  <build>
    <pluginManagement>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-compiler-plugin</artifactId>
          <version>3.1</version>
          <configuration>
            <source>${java.version}</source>
            <target>${java.version}</target>
          </configuration>
        </plugin>
      </plugins>
    </pluginManagement>
  </build>
</project>
```



The `spark-core` package is marked as provided in case we package our application into an assembly JAR. This is covered in more detail in [Chapter 7](#).

Once we have our build defined, we can easily package and run our application using the `bin/spark-submit` script. The `spark-submit` script sets up a number of environment variables used by Spark. From the *mini-complete-example* directory we can build in both Scala ([Example 2-14](#)) and Java ([Example 2-15](#)).

#### *Example 2-14. Scala build and run*

```
sbt clean package
$SPARK_HOME/bin/spark-submit \
  --class com.oreilly.learningsparkexamples.mini.scala.WordCount \
  ./target/...(as above) \
  ./README.md ./wordcounts
```

#### *Example 2-15. Maven build and run*

```
mvn clean && mvn compile && mvn package
$SPARK_HOME/bin/spark-submit \
  --class com.oreilly.learningsparkexamples.mini.java.WordCount \
  ./target/learning-spark-mini-example-0.0.1.jar \
  ./README.md ./wordcounts
```

For even more detailed examples of linking applications to Spark, refer to the [Quick Start Guide](#) in the official Spark documentation. [Chapter 7](#) covers packaging Spark applications in more detail.

## Conclusion

In this chapter, we have covered downloading Spark, running it locally on your laptop, and using it either interactively or from a standalone application. We gave a quick overview of the core concepts involved in programming with Spark: a driver program creates a `SparkContext` and RDDs, and then runs parallel operations on them. In the next chapter, we will dive more deeply into how RDDs operate.



---

# Programming with RDDs

This chapter introduces Spark's core abstraction for working with data, the resilient distributed dataset (RDD). An RDD is simply a distributed collection of elements. In Spark all work is expressed as either creating new RDDs, transforming existing RDDs, or calling operations on RDDs to compute a result. Under the hood, Spark automatically distributes the data contained in RDDs across your cluster and parallelizes the operations you perform on them.

Both data scientists and engineers should read this chapter, as RDDs are the core concept in Spark. We highly recommend that you try some of these examples in an interactive shell (see [“Introduction to Spark's Python and Scala Shells” on page 11](#)). In addition, all code in this chapter is available in the book's [GitHub repository](#).

## RDD Basics

An RDD in Spark is simply an immutable distributed collection of objects. Each RDD is split into multiple *partitions*, which may be computed on different nodes of the cluster. RDDs can contain any type of Python, Java, or Scala objects, including user-defined classes.

Users create RDDs in two ways: by loading an external dataset, or by distributing a collection of objects (e.g., a list or set) in their driver program. We have already seen loading a text file as an RDD of strings using `SparkContext.textFile()`, as shown in [Example 3-1](#).

*Example 3-1. Creating an RDD of strings with `textFile()` in Python*

```
>>> lines = sc.textFile("README.md")
```

Once created, RDDs offer two types of operations: *transformations* and *actions*. *Transformations* construct a new RDD from a previous one. For example, one common transformation is filtering data that matches a predicate. In our text file example, we can use this to create a new RDD holding just the strings that contain the word *Python*, as shown in [Example 3-2](#).

*Example 3-2. Calling the filter() transformation*

```
>>> pythonLines = lines.filter(lambda line: "Python" in line)
```

*Actions*, on the other hand, compute a result based on an RDD, and either return it to the driver program or save it to an external storage system (e.g., HDFS). One example of an action we called earlier is `first()`, which returns the first element in an RDD and is demonstrated in [Example 3-3](#).

*Example 3-3. Calling the first() action*

```
>>> pythonLines.first()
u'## Interactive Python Shell'
```

Transformations and actions are different because of the way Spark computes RDDs. Although you can define new RDDs any time, Spark computes them only in a *lazy* fashion—that is, the first time they are used in an action. This approach might seem unusual at first, but makes a lot of sense when you are working with Big Data. For instance, consider [Example 3-2](#) and [Example 3-3](#), where we defined a text file and then filtered the lines that include *Python*. If Spark were to load and store all the lines in the file as soon as we wrote `lines = sc.textFile(...)`, it would waste a lot of storage space, given that we then immediately filter out many lines. Instead, once Spark sees the whole chain of transformations, it can compute just the data needed for its result. In fact, for the `first()` action, Spark scans the file only until it finds the first matching line; it doesn't even read the whole file.

Finally, Spark's RDDs are by default recomputed each time you run an action on them. If you would like to reuse an RDD in multiple actions, you can ask Spark to *persist* it using `RDD.persist()`. We can ask Spark to persist our data in a number of different places, which will be covered in [Table 3-6](#). After computing it the first time, Spark will store the RDD contents in memory (partitioned across the machines in your cluster), and reuse them in future actions. Persisting RDDs on disk instead of memory is also possible. The behavior of not persisting by default may again seem unusual, but it makes a lot of sense for big datasets: if you will not reuse the RDD,

there's no reason to waste storage space when Spark could instead stream through the data once and just compute the result.<sup>1</sup>

In practice, you will often use `persist()` to load a subset of your data into memory and query it repeatedly. For example, if we knew that we wanted to compute multiple results about the README lines that contain *Python*, we could write the script shown in [Example 3-4](#).

*Example 3-4. Persisting an RDD in memory*

```
>>> pythonLines.persist()

>>> pythonLines.count()
2

>>> pythonLines.first()
u'## Interactive Python Shell'
```

To summarize, every Spark program and shell session will work as follows:

1. Create some input RDDs from external data.
2. Transform them to define new RDDs using transformations like `filter()`.
3. Ask Spark to `persist()` any intermediate RDDs that will need to be reused.
4. Launch actions such as `count()` and `first()` to kick off a parallel computation, which is then optimized and executed by Spark.



`cache()` is the same as calling `persist()` with the default storage level.

In the rest of this chapter, we'll go through each of these steps in detail, and cover some of the most common RDD operations in Spark.

## Creating RDDs

Spark provides two ways to create RDDs: loading an external dataset and parallelizing a collection in your driver program.

---

<sup>1</sup> The ability to always recompute an RDD is actually why RDDs are called “resilient.” When a machine holding RDD data fails, Spark uses this ability to recompute the missing partitions, transparent to the user.

The simplest way to create RDDs is to take an existing collection in your program and pass it to SparkContext's `parallelize()` method, as shown in Examples 3-5 through 3-7. This approach is very useful when you are learning Spark, since you can quickly create your own RDDs in the shell and perform operations on them. Keep in mind, however, that outside of prototyping and testing, this is not widely used since it requires that you have your entire dataset in memory on one machine.

*Example 3-5. `parallelize()` method in Python*

```
lines = sc.parallelize(["pandas", "i like pandas"])
```

*Example 3-6. `parallelize()` method in Scala*

```
val lines = sc.parallelize(List("pandas", "i like pandas"))
```

*Example 3-7. `parallelize()` method in Java*

```
JavaRDD<String> lines = sc.parallelize(Arrays.asList("pandas", "i like pandas"));
```

A more common way to create RDDs is to load data from external storage. Loading external datasets is covered in detail in Chapter 5. However, we already saw one method that loads a text file as an RDD of strings, `SparkContext.textFile()`, which is shown in Examples 3-8 through 3-10.

*Example 3-8. `textFile()` method in Python*

```
lines = sc.textFile("/path/to/README.md")
```

*Example 3-9. `textFile()` method in Scala*

```
val lines = sc.textFile("/path/to/README.md")
```

*Example 3-10. `textFile()` method in Java*

```
JavaRDD<String> lines = sc.textFile("/path/to/README.md");
```

## RDD Operations

As we've discussed, RDDs support two types of operations: *transformations* and *actions*. Transformations are operations on RDDs that return a new RDD, such as `map()` and `filter()`. Actions are operations that return a result to the driver program or write it to storage, and kick off a computation, such as `count()` and `first()`. Spark treats transformations and actions very differently, so understanding which type of operation you are performing will be important. If you are ever confused

whether a given function is a transformation or an action, you can look at its return type: transformations return RDDs, whereas actions return some other data type.

## Transformations

Transformations are operations on RDDs that return a new RDD. As discussed in “[Lazy Evaluation](#)” on page 29, transformed RDDs are computed lazily, only when you use them in an action. Many transformations are *element-wise*; that is, they work on one element at a time; but this is not true for all transformations.

As an example, suppose that we have a logfile, *log.txt*, with a number of messages, and we want to select only the error messages. We can use the `filter()` transformation seen before. This time, though, we’ll show a filter in all three of Spark’s language APIs (Examples 3-11 through 3-13).

*Example 3-11. filter() transformation in Python*

```
inputRDD = sc.textFile("log.txt")
errorsRDD = inputRDD.filter(lambda x: "error" in x)
```

*Example 3-12. filter() transformation in Scala*

```
val inputRDD = sc.textFile("log.txt")
val errorsRDD = inputRDD.filter(line => line.contains("error"))
```

*Example 3-13. filter() transformation in Java*

```
JavaRDD<String> inputRDD = sc.textFile("log.txt");
JavaRDD<String> errorsRDD = inputRDD.filter(
    new Function<String, Boolean>() {
        public Boolean call(String x) { return x.contains("error"); }
    }
);
```

Note that the `filter()` operation does not mutate the existing `inputRDD`. Instead, it returns a pointer to an entirely new RDD. `inputRDD` can still be reused later in the program—for instance, to search for other words. In fact, let’s use `inputRDD` again to search for lines with the word *warning* in them. Then, we’ll use another transformation, `union()`, to print out the number of lines that contained either *error* or *warning*. We show Python in [Example 3-14](#), but the `union()` function is identical in all three languages.

### Example 3-14. `union()` transformation in Python

```
errorsRDD = inputRDD.filter(lambda x: "error" in x)
warningsRDD = inputRDD.filter(lambda x: "warning" in x)
badLinesRDD = errorsRDD.union(warningsRDD)
```

`union()` is a bit different than `filter()`, in that it operates on two RDDs instead of one. Transformations can actually operate on any number of input RDDs.



A better way to accomplish the same result as in [Example 3-14](#) would be to simply filter the `inputRDD` once, looking for either *error* or *warning*.

Finally, as you derive new RDDs from each other using transformations, Spark keeps track of the set of dependencies between different RDDs, called the *lineage graph*. It uses this information to compute each RDD on demand and to recover lost data if part of a persistent RDD is lost. [Figure 3-1](#) shows a lineage graph for [Example 3-14](#).

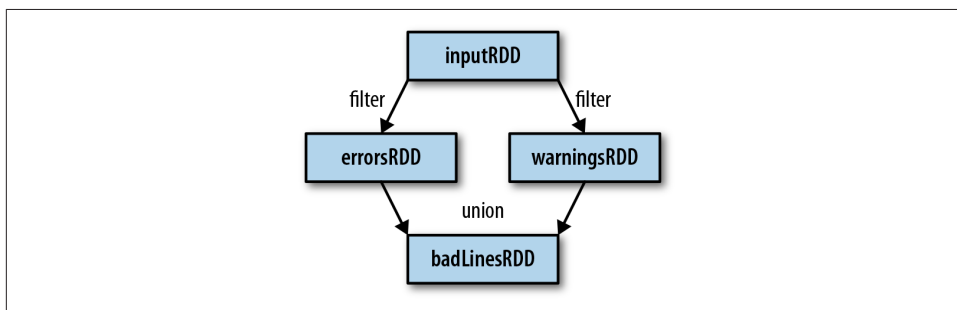


Figure 3-1. RDD lineage graph created during log analysis

## Actions

We've seen how to create RDDs from each other with transformations, but at some point, we'll want to actually *do something* with our dataset. Actions are the second type of RDD operation. They are the operations that return a final value to the driver program or write data to an external storage system. Actions force the evaluation of the transformations required for the RDD they were called on, since they need to actually produce output.

Continuing the log example from the previous section, we might want to print out some information about the `badLinesRDD`. To do that, we'll use two actions, `count()`, which returns the count as a number, and `take()`, which collects a number of elements from the RDD, as shown in [Examples 3-15](#) through [3-17](#).

### Example 3-15. Python error count using actions

```
print "Input had " + badLinesRDD.count() + " concerning lines"
print "Here are 10 examples:"
for line in badLinesRDD.take(10):
    print line
```

### Example 3-16. Scala error count using actions

```
println("Input had " + badLinesRDD.count() + " concerning lines")
println("Here are 10 examples:")
badLinesRDD.take(10).foreach(println)
```

### Example 3-17. Java error count using actions

```
System.out.println("Input had " + badLinesRDD.count() + " concerning lines")
System.out.println("Here are 10 examples:")
for (String line: badLinesRDD.take(10)) {
    System.out.println(line);
}
```

In this example, we used `take()` to retrieve a small number of elements in the RDD at the driver program. We then iterate over them locally to print out information at the driver. RDDs also have a `collect()` function to retrieve the entire RDD. This can be useful if your program filters RDDs down to a very small size and you'd like to deal with it locally. Keep in mind that your entire dataset must fit in memory on a single machine to use `collect()` on it, so `collect()` shouldn't be used on large datasets.

In most cases RDDs can't just be `collect()`ed to the driver because they are too large. In these cases, it's common to write data out to a distributed storage system such as HDFS or Amazon S3. You can save the contents of an RDD using the `saveAsTextFile()` action, `saveAsSequenceFile()`, or any of a number of actions for various built-in formats. We will cover the different options for exporting data in [Chapter 5](#).

It is important to note that each time we call a new action, the entire RDD must be computed "from scratch." To avoid this inefficiency, users can *persist* intermediate results, as we will cover in ["Persistence \(Caching\)" on page 44](#).

## Lazy Evaluation

As you read earlier, transformations on RDDs are lazily evaluated, meaning that Spark will not begin to execute until it sees an action. This can be somewhat counter-intuitive for new users, but may be familiar for those who have used functional languages such as Haskell or LINQ-like data processing frameworks.

Lazy evaluation means that when we call a transformation on an RDD (for instance, calling `map()`), the operation is not immediately performed. Instead, Spark internally records metadata to indicate that this operation has been requested. Rather than thinking of an RDD as containing specific data, it is best to think of each RDD as consisting of instructions on how to compute the data that we build up through transformations. Loading data into an RDD is lazily evaluated in the same way transformations are. So, when we call `sc.textFile()`, the data is not loaded until it is necessary. As with transformations, the operation (in this case, reading the data) can occur multiple times.



Although transformations are lazy, you can force Spark to execute them at any time by running an action, such as `count()`. This is an easy way to test out just part of your program.

Spark uses lazy evaluation to reduce the number of passes it has to take over our data by grouping operations together. In systems like Hadoop MapReduce, developers often have to spend a lot of time considering how to group together operations to minimize the number of MapReduce passes. In Spark, there is no substantial benefit to writing a single complex map instead of chaining together many simple operations. Thus, users are free to organize their program into smaller, more manageable operations.

## Passing Functions to Spark

Most of Spark's transformations, and some of its actions, depend on passing in functions that are used by Spark to compute data. Each of the core languages has a slightly different mechanism for passing functions to Spark.

### Python

In Python, we have three options for passing functions into Spark. For shorter functions, we can pass in lambda expressions, as we did in [Example 3-2](#), and as [Example 3-18](#) demonstrates. Alternatively, we can pass in top-level functions, or locally defined functions.

*Example 3-18. Passing functions in Python*

```
word = rdd.filter(lambda s: "error" in s)
def containsError(s):
    return "error" in s
word = rdd.filter(containsError)
```

One issue to watch out for when passing functions is inadvertently serializing the object containing the function. When you pass a function that is the member of an object, or contains references to fields in an object (e.g., `self.field`), Spark sends the *entire* object to worker nodes, which can be much larger than the bit of information you need (see [Example 3-19](#)). Sometimes this can also cause your program to fail, if your class contains objects that Python can't figure out how to pickle.

*Example 3-19. Passing a function with field references (don't do this!)*

```
class SearchFunctions(object):
    def __init__(self, query):
        self.query = query
    def isMatch(self, s):
        return self.query in s
    def getMatchesFunctionReference(self, rdd):
        # Problem: references all of "self" in "self.isMatch"
        return rdd.filter(self.isMatch)
    def getMatchesMemberReference(self, rdd):
        # Problem: references all of "self" in "self.query"
        return rdd.filter(lambda x: self.query in x)
```

Instead, just extract the fields you need from your object into a local variable and pass that in, like we do in [Example 3-20](#).

*Example 3-20. Python function passing without field references*

```
class WordFunctions(object):
    ...
    def getMatchesNoReference(self, rdd):
        # Safe: extract only the field we need into a local variable
        query = self.query
        return rdd.filter(lambda x: query in x)
```

## Scala

In Scala, we can pass in functions defined inline, references to methods, or static functions as we do for Scala's other functional APIs. Some other considerations come into play, though—namely that the function we pass and the data referenced in it needs to be serializable (implementing Java's `Serializable` interface). Furthermore, as in Python, passing a method or field of an object includes a reference to that whole object, though this is less obvious because we are not forced to write these references with `self`. As we did with Python in [Example 3-20](#), we can instead extract the fields we need as local variables and avoid needing to pass the whole object containing them, as shown in [Example 3-21](#).

### Example 3-21. Scala function passing

```
class SearchFunctions(val query: String) {
  def isMatch(s: String): Boolean = {
    s.contains(query)
  }
  def getMatchesFunctionReference(rdd: RDD[String]): RDD[String] = {
    // Problem: "isMatch" means "this.isMatch", so we pass all of "this"
    rdd.map(isMatch)
  }
  def getMatchesFieldReference(rdd: RDD[String]): RDD[String] = {
    // Problem: "query" means "this.query", so we pass all of "this"
    rdd.map(x => x.split(query))
  }
  def getMatchesNoReference(rdd: RDD[String]): RDD[String] = {
    // Safe: extract just the field we need into a local variable
    val query_ = this.query
    rdd.map(x => x.split(query_))
  }
}
```

If `NotSerializableException` occurs in Scala, a reference to a method or field in a nonserializable class is usually the problem. Note that passing in local serializable variables or functions that are members of a top-level object is always safe.

## Java

In Java, functions are specified as objects that implement one of Spark's function interfaces from the `org.apache.spark.api.java.function` package. There are a number of different interfaces based on the return type of the function. We show the most basic function interfaces in [Table 3-1](#), and cover a number of **other function interfaces** for when we need to return special types of data, like key/value data, in [“Java” on page 43](#).

Table 3-1. Standard Java function interfaces

| Function name                            | Method to implement                    | Usage                                                                                                                    |
|------------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>Function&lt;T, R&gt;</code>        | <code>R call(T)</code>                 | Take in one input and return one output, for use with operations like <code>map()</code> and <code>filter()</code> .     |
| <code>Function2&lt;T1, T2, R&gt;</code>  | <code>R call(T1, T2)</code>            | Take in two inputs and return one output, for use with operations like <code>aggregate()</code> or <code>fold()</code> . |
| <code>FlatMapFunction&lt;T, R&gt;</code> | <code>Iterable&lt;R&gt; call(T)</code> | Take in one input and return zero or more outputs, for use with operations like <code>flatMap()</code> .                 |

We can either define our function classes inline as anonymous inner classes ([Example 3-22](#)), or create a named class ([Example 3-23](#)).

*Example 3-22. Java function passing with anonymous inner class*

```
RDD<String> errors = lines.filter(new Function<String, Boolean>() {  
    public Boolean call(String x) { return x.contains("error"); }  
});
```

*Example 3-23. Java function passing with named class*

```
class ContainsError implements Function<String, Boolean>() {  
    public Boolean call(String x) { return x.contains("error"); }  
}  
  
RDD<String> errors = lines.filter(new ContainsError());
```

The style to choose is a personal preference, but we find that top-level named functions are often cleaner for organizing large programs. One other benefit of top-level functions is that you can give them constructor parameters, as shown in [Example 3-24](#).

*Example 3-24. Java function class with parameters*

```
class Contains implements Function<String, Boolean>() {  
    private String query;  
    public Contains(String query) { this.query = query; }  
    public Boolean call(String x) { return x.contains(query); }  
}  
  
RDD<String> errors = lines.filter(new Contains("error"));
```

In Java 8, you can also use lambda expressions to concisely implement the function interfaces. Since Java 8 is still relatively new as of this writing, our examples use the more verbose syntax for defining classes in previous versions of Java. However, with lambda expressions, our search example would look like [Example 3-25](#).

*Example 3-25. Java function passing with lambda expression in Java 8*

```
RDD<String> errors = lines.filter(s -> s.contains("error"));
```

If you are interested in using Java 8's lambda expression, refer to [Oracle's documentation](#) and [the Databricks blog post on how to use lambdas with Spark](#).



Both anonymous inner classes and lambda expressions can reference any `final` variables in the method enclosing them, so you can pass these variables to Spark just as in Python and Scala.

## Common Transformations and Actions

In this chapter, we tour the most common transformations and actions in Spark. Additional operations are available on RDDs containing certain types of data—for example, statistical functions on RDDs of numbers, and key/value operations such as aggregating data by key on RDDs of key/value pairs. We cover converting between RDD types and these special operations in later sections.

### Basic RDDs

We will begin by describing what transformations and actions we can perform on all RDDs regardless of the data.

#### Element-wise transformations

The two most common transformations you will likely be using are `map()` and `filter()` (see [Figure 3-2](#)). The `map()` transformation takes in a function and applies it to each element in the RDD with the result of the function being the new value of each element in the resulting RDD. The `filter()` transformation takes in a function and returns an RDD that only has elements that pass the `filter()` function.

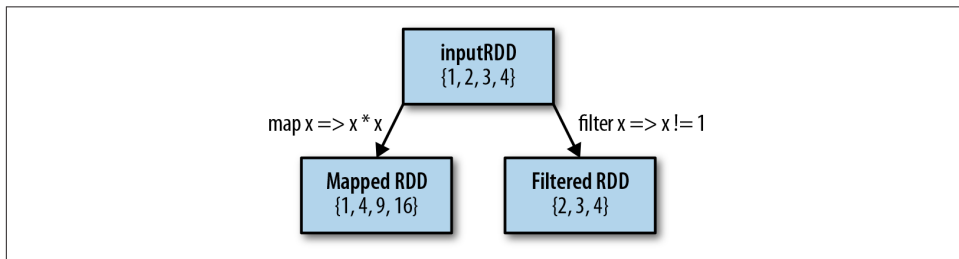


Figure 3-2. Mapped and filtered RDD from an input RDD

We can use `map()` to do any number of things, from fetching the website associated with each URL in our collection to just squaring the numbers. It is useful to note that `map()`'s return type does not have to be the same as its input type, so if we had an RDD `String` and our `map()` function were to parse the strings and return a `Double`, our input RDD type would be `RDD[String]` and the resulting RDD type would be `RDD[Double]`.

Let's look at a basic example of `map()` that squares all of the numbers in an RDD (Examples 3-26 through 3-28).

*Example 3-26. Python squaring the values in an RDD*

```
nums = sc.parallelize([1, 2, 3, 4])
squared = nums.map(lambda x: x * x).collect()
for num in squared:
    print "%i " % (num)
```

*Example 3-27. Scala squaring the values in an RDD*

```
val input = sc.parallelize(List(1, 2, 3, 4))
val result = input.map(x => x * x)
println(result.collect().mkString(", "))
```

*Example 3-28. Java squaring the values in an RDD*

```
JavaRDD<Integer> rdd = sc.parallelize(Arrays.asList(1, 2, 3, 4));
JavaRDD<Integer> result = rdd.map(new Function<Integer, Integer>() {
    public Integer call(Integer x) { return x*x; }
});
System.out.println(StringUtils.join(result.collect(), ", "));
```

Sometimes we want to produce multiple output elements for each input element. The operation to do this is called `flatMap()`. As with `map()`, the function we provide to `flatMap()` is called individually for each element in our input RDD. Instead of returning a single element, we return an iterator with our return values. Rather than producing an RDD of iterators, we get back an RDD that consists of the elements from all of the iterators. A simple usage of `flatMap()` is splitting up an input string into words, as shown in Examples 3-29 through 3-31.

*Example 3-29. flatMap() in Python, splitting lines into words*

```
lines = sc.parallelize(["hello world", "hi"])
words = lines.flatMap(lambda line: line.split(" "))
words.first() # returns "hello"
```

*Example 3-30. flatMap() in Scala, splitting lines into multiple words*

```
val lines = sc.parallelize(List("hello world", "hi"))
val words = lines.flatMap(line => line.split(" "))
words.first() // returns "hello"
```

Example 3-31. `flatMap()` in Java, splitting lines into multiple words

```
JavaRDD<String> lines = sc.parallelize(Arrays.asList("hello world", "hi"));
JavaRDD<String> words = lines.flatMap(new FlatMapFunction<String, String>() {
    public Iterable<String> call(String line) {
        return Arrays.asList(line.split(" "));
    }
});
words.first(); // returns "hello"
```

We illustrate the difference between `flatMap()` and `map()` in Figure 3-3. You can think of `flatMap()` as “flattening” the iterators returned to it, so that instead of ending up with an RDD of lists we have an RDD of the elements in those lists.

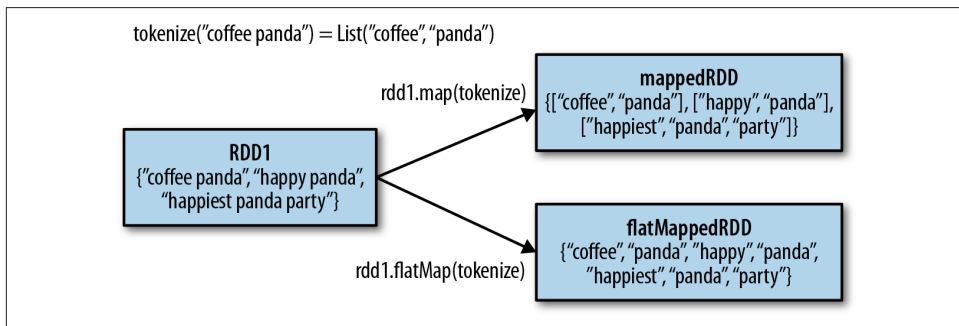


Figure 3-3. Difference between `flatMap()` and `map()` on an RDD

## Pseudo set operations

RDDs support many of the operations of mathematical sets, such as union and intersection, even when the RDDs themselves are not properly sets. Four operations are shown in Figure 3-4. It’s important to note that all of these operations require that the RDDs being operated on are of the same type.

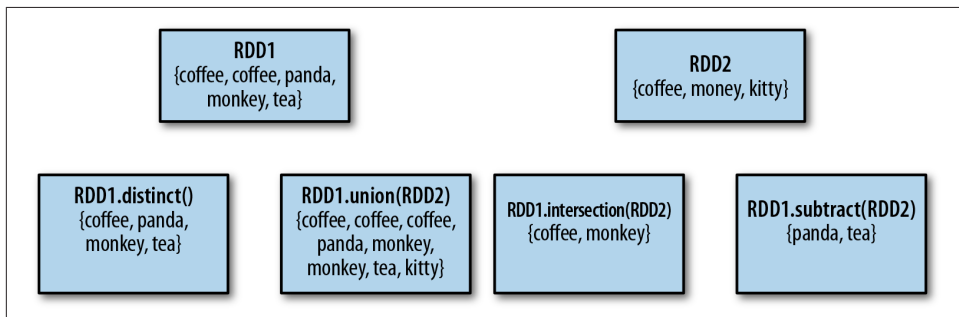


Figure 3-4. Some simple set operations

The set property most frequently missing from our RDDs is the uniqueness of elements, as we often have duplicates. If we want only unique elements we can use the `RDD.distinct()` transformation to produce a new RDD with only distinct items. Note that `distinct()` is expensive, however, as it requires shuffling all the data over the network to ensure that we receive only one copy of each element. Shuffling, and how to avoid it, is discussed in more detail in [Chapter 4](#).

The simplest set operation is `union(other)`, which gives back an RDD consisting of the data from both sources. This can be useful in a number of use cases, such as processing logfiles from many sources. Unlike the mathematical `union()`, if there are duplicates in the input RDDs, the result of Spark's `union()` will contain duplicates (which we can fix if desired with `distinct()`).

Spark also provides an `intersection(other)` method, which returns only elements in both RDDs. `intersection()` also removes all duplicates (including duplicates from a single RDD) while running. While `intersection()` and `union()` are two similar concepts, the performance of `intersection()` is much worse since it requires a shuffle over the network to identify common elements.

Sometimes we need to remove some data from consideration. The `subtract(other)` function takes in another RDD and returns an RDD that has only values present in the first RDD and not the second RDD. Like `intersection()`, it performs a shuffle.

We can also compute a Cartesian product between two RDDs, as shown in [Figure 3-5](#). The `cartesian(other)` transformation returns all possible pairs of (a, b) where a is in the source RDD and b is in the other RDD. The Cartesian product can be useful when we wish to consider the similarity between all possible pairs, such as computing every user's expected interest in each offer. We can also take the Cartesian product of an RDD with itself, which can be useful for tasks like user similarity. Be warned, however, that the Cartesian product is *very expensive* for large RDDs.

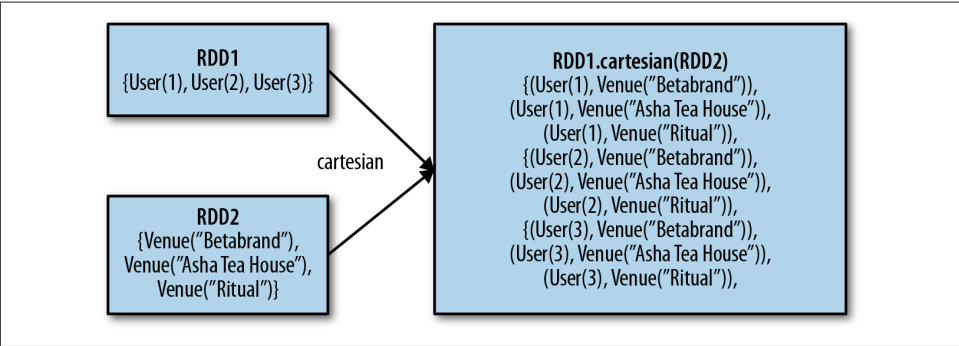


Figure 3-5. Cartesian product between two RDDs

Tables [3-2](#) and [3-3](#) summarize these and other common RDD transformations.

Table 3-2. Basic RDD transformations on an RDD containing {1, 2, 3, 3}

| Function name                                          | Purpose                                                                                                                               | Example                                   | Result                |
|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|-----------------------|
| <code>map()</code>                                     | Apply a function to each element in the RDD and return an RDD of the result.                                                          | <code>rdd.map(x =&gt; x + 1)</code>       | {2, 3, 4, 4}          |
| <code>flatMap()</code>                                 | Apply a function to each element in the RDD and return an RDD of the contents of the iterators returned. Often used to extract words. | <code>rdd.flatMap(x =&gt; x.to(3))</code> | {1, 2, 3, 2, 3, 3, 3} |
| <code>filter()</code>                                  | Return an RDD consisting of only elements that pass the condition passed to <code>filter()</code> .                                   | <code>rdd.filter(x =&gt; x != 1)</code>   | {2, 3, 3}             |
| <code>distinct()</code>                                | Remove duplicates.                                                                                                                    | <code>rdd.distinct()</code>               | {1, 2, 3}             |
| <code>sample(withReplacement, fraction, [seed])</code> | Sample an RDD, with or without replacement.                                                                                           | <code>rdd.sample(false, 0.5)</code>       | Nondeterministic      |

Table 3-3. Two-RDD transformations on RDDs containing {1, 2, 3} and {3, 4, 5}

| Function name               | Purpose                                                      | Example                              | Result                      |
|-----------------------------|--------------------------------------------------------------|--------------------------------------|-----------------------------|
| <code>union()</code>        | Produce an RDD containing elements from both RDDs.           | <code>rdd.union(other)</code>        | {1, 2, 3, 3, 4, 5}          |
| <code>intersection()</code> | RDD containing only elements found in both RDDs.             | <code>rdd.intersection(other)</code> | {3}                         |
| <code>subtract()</code>     | Remove the contents of one RDD (e.g., remove training data). | <code>rdd.subtract(other)</code>     | {1, 2}                      |
| <code>cartesian()</code>    | Cartesian product with the other RDD.                        | <code>rdd.cartesian(other)</code>    | {(1, 3), (1, 4), ... (3,5)} |

## Actions

The most common action on basic RDDs you will likely use is `reduce()`, which takes a function that operates on two elements of the type in your RDD and returns a new element of the same type. A simple example of such a function is `+`, which we can use to sum our RDD. With `reduce()`, we can easily sum the elements of our RDD, count the number of elements, and perform other types of aggregations (see Examples 3-32 through 3-34).

### Example 3-32. `reduce()` in Python

```
sum = rdd.reduce(lambda x, y: x + y)
```

### Example 3-33. `reduce()` in Scala

```
val sum = rdd.reduce((x, y) => x + y)
```

### Example 3-34. `reduce()` in Java

```
Integer sum = rdd.reduce(new Function2<Integer, Integer, Integer>() {  
    public Integer call(Integer x, Integer y) { return x + y; }  
});
```

Similar to `reduce()` is `fold()`, which also takes a function with the same signature as needed for `reduce()`, but in addition takes a “zero value” to be used for the initial call on each partition. The zero value you provide should be the identity element for your operation; that is, applying it multiple times with your function should not change the value (e.g., 0 for +, 1 for \*, or an empty list for concatenation).



You can minimize object creation in `fold()` by modifying and returning the first of the two parameters in place. However, you should not modify the second parameter.

Both `fold()` and `reduce()` require that the return type of our result be the same type as that of the elements in the RDD we are operating over. This works well for operations like `sum`, but sometimes we want to return a different type. For example, when computing a running average, we need to keep track of both the count so far and the number of elements, which requires us to return a pair. We could work around this by first using `map()` where we transform every element into the element and the number 1, which is the type we want to return, so that the `reduce()` function can work on pairs.

The `aggregate()` function frees us from the constraint of having the return be the same type as the RDD we are working on. With `aggregate()`, like `fold()`, we supply an initial zero value of the type we want to return. We then supply a function to combine the elements from our RDD with the accumulator. Finally, we need to supply a second function to merge two accumulators, given that each node accumulates its own results locally.

We can use `aggregate()` to compute the average of an RDD, avoiding a `map()` before the `fold()`, as shown in Examples 3-35 through 3-37.

*Example 3-35. aggregate() in Python*

```
sumCount = nums.aggregate((0, 0),
    (lambda acc, value: (acc[0] + value, acc[1] + 1),
    (lambda acc1, acc2: (acc1[0] + acc2[0], acc1[1] + acc2[1]))))
return sumCount[0] / float(sumCount[1])
```

*Example 3-36. aggregate() in Scala*

```
val result = input.aggregate((0, 0))(
    (acc, value) => (acc._1 + value, acc._2 + 1),
    (acc1, acc2) => (acc1._1 + acc2._1, acc1._2 + acc2._2))
val avg = result._1 / result._2.toDouble
```

*Example 3-37. aggregate() in Java*

```
class AvgCount implements Serializable {
    public AvgCount(int total, int num) {
        this.total = total;
        this.num = num;
    }
    public int total;
    public int num;
    public double avg() {
        return total / (double) num;
    }
}

Function2<AvgCount, Integer, AvgCount> addAndCount =
    new Function2<AvgCount, Integer, AvgCount>() {
        public AvgCount call(AvgCount a, Integer x) {
            a.total += x;
            a.num += 1;
            return a;
        }
    };

Function2<AvgCount, AvgCount, AvgCount> combine =
    new Function2<AvgCount, AvgCount, AvgCount>() {
        public AvgCount call(AvgCount a, AvgCount b) {
            a.total += b.total;
            a.num += b.num;
            return a;
        }
    };

AvgCount initial = new AvgCount(0, 0);
AvgCount result = rdd.aggregate(initial, addAndCount, combine);
System.out.println(result.avg());
```

Some actions on RDDs return some or all of the data to our driver program in the form of a regular collection or value.

The simplest and most common operation that returns data to our driver program is `collect()`, which returns the entire RDD's contents. `collect()` is commonly used in unit tests where the entire contents of the RDD are expected to fit in memory, as that makes it easy to compare the value of our RDD with our expected result. `collect()` suffers from the restriction that all of your data must fit on a single machine, as it all needs to be copied to the driver.

`take(n)` returns  $n$  elements from the RDD and attempts to minimize the number of partitions it accesses, so it may represent a biased collection. It's important to note that these operations do not return the elements in the order you might expect.

These operations are useful for unit tests and quick debugging, but may introduce bottlenecks when you're dealing with large amounts of data.

If there is an ordering defined on our data, we can also extract the top elements from an RDD using `top()`. `top()` will use the default ordering on the data, but we can supply our own comparison function to extract the top elements.

Sometimes we need a sample of our data in our driver program. The `takeSample(withReplacement, num, seed)` function allows us to take a sample of our data either with or without replacement.

Sometimes it is useful to perform an action on all of the elements in the RDD, but without returning any result to the driver program. A good example of this would be posting JSON to a webserver or inserting records into a database. In either case, the `foreach()` action lets us perform computations on each element in the RDD without bringing it back locally.

The further standard operations on a basic RDD all behave pretty much exactly as you would imagine from their name. `count()` returns a count of the elements, and `countByValue()` returns a map of each unique value to its count. Table 3-4 summarizes these and other actions.

Table 3-4. Basic actions on an RDD containing {1, 2, 3, 3}

| Function name               | Purpose                                         | Example                         | Result                         |
|-----------------------------|-------------------------------------------------|---------------------------------|--------------------------------|
| <code>collect()</code>      | Return all elements from the RDD.               | <code>rdd.collect()</code>      | {1, 2, 3, 3}                   |
| <code>count()</code>        | Number of elements in the RDD.                  | <code>rdd.count()</code>        | 4                              |
| <code>countByValue()</code> | Number of times each element occurs in the RDD. | <code>rdd.countByValue()</code> | {(1, 1),<br>(2, 1),<br>(3, 2)} |

| Function name                                         | Purpose                                                               | Example                                                                                                                         | Result           |
|-------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------|
| <code>take(num)</code>                                | Return num elements from the RDD.                                     | <code>rdd.take(2)</code>                                                                                                        | {1, 2}           |
| <code>top(num)</code>                                 | Return the top num elements the RDD.                                  | <code>rdd.top(2)</code>                                                                                                         | {3, 3}           |
| <code>takeOrdered(num)(ordering)</code>               | Return num elements based on provided ordering.                       | <code>rdd.takeOrdered(2)(myOrdering)</code>                                                                                     | {3, 3}           |
| <code>takeSample(withReplacement, num, [seed])</code> | Return num elements at random.                                        | <code>rdd.takeSample(false, 1)</code>                                                                                           | Nondeterministic |
| <code>reduce(func)</code>                             | Combine the elements of the RDD together in parallel (e.g., sum).     | <code>rdd.reduce((x, y) =&gt; x + y)</code>                                                                                     | 9                |
| <code>fold(zero)(func)</code>                         | Same as <code>reduce()</code> but with the provided zero value.       | <code>rdd.fold(0)((x, y) =&gt; x + y)</code>                                                                                    | 9                |
| <code>aggregate(zeroValue)(seqOp, combOp)</code>      | Similar to <code>reduce()</code> but used to return a different type. | <code>rdd.aggregate((0, 0))<br/>((x, y) =&gt;<br/>(x._1 + y, x._2 + 1),<br/>(x, y) =&gt;<br/>(x._1 + y._1, x._2 + y._2))</code> | (9, 4)           |
| <code>foreach(func)</code>                            | Apply the provided function to each element of the RDD.               | <code>rdd.foreach(func)</code>                                                                                                  | Nothing          |

## Converting Between RDD Types

Some functions are available only on certain types of RDDs, such as `mean()` and `variance()` on numeric RDDs or `join()` on key/value pair RDDs. We will cover these special functions for **numeric data** in [Chapter 6](#) and pair RDDs in [Chapter 4](#). In Scala and Java, these methods aren't defined on the standard RDD class, so to access this additional functionality we have to make sure we get the correct specialized class.

## Scala

In Scala the conversion to RDDs with special functions (e.g., to expose numeric functions on an `RDD[Double]`) is handled automatically using implicit conversions. As mentioned in “[Initializing a SparkContext](#)” on page 17, we need to add `import org.apache.spark.SparkContext._` for these conversions to work. You can see the implicit conversions listed in the [SparkContext object’s ScalaDoc](#). These implicits turn an RDD into various wrapper classes, such as `DoubleRDDFunctions` (for RDDs of numeric data) and `PairRDDFunctions` (for key/value pairs), to expose additional functions such as `mean()` and `variance()`.

Implicits, while quite powerful, can sometimes be confusing. If you call a function like `mean()` on an RDD, you might look at the [Scaladocs for the RDD class](#) and notice there is no `mean()` function. The call manages to succeed because of implicit conversions between `RDD[Double]` and `DoubleRDDFunctions`. When searching for functions on your RDD in Scaladoc, make sure to look at functions that are available in these wrapper classes.

## Java

In Java the conversion between the specialized types of RDDs is a bit more explicit. In particular, there are special classes called `JavaDoubleRDD` and `JavaPairRDD` for RDDs of these types, with extra methods for these types of data. This has the benefit of giving you a greater understanding of what exactly is going on, but can be a bit more cumbersome.

To construct RDDs of these special types, instead of always using the `Function` class we will need to use specialized versions. If we want to create a `DoubleRDD` from an RDD of type `T`, rather than using `Function<T, Double>` we use `DoubleFunction<T>`. [Table 3-5](#) shows the specialized functions and their uses.

We also need to call different functions on our RDD (so we can’t just create a `DoubleFunction` and pass it to `map()`). When we want a `DoubleRDD` back, instead of calling `map()`, we need to call `mapToDouble()` with the same pattern all of the other functions follow.

*Table 3-5. Java interfaces for type-specific functions*

| Function name                               | Equivalent function* <code>&lt;A, B,...&gt;</code>     | Usage                                                      |
|---------------------------------------------|--------------------------------------------------------|------------------------------------------------------------|
| <code>DoubleFlatMapFunction&lt;T&gt;</code> | <code>Function&lt;T, Iterable&lt;Double&gt;&gt;</code> | <code>DoubleRDD</code> from a <code>flatMapToDouble</code> |
| <code>DoubleFunction&lt;T&gt;</code>        | <code>Function&lt;T, double&gt;</code>                 | <code>DoubleRDD</code> from <code>mapToDouble</code>       |

| Function name                | Equivalent function*<A, B,...>      | Usage                                    |
|------------------------------|-------------------------------------|------------------------------------------|
| PairFlatMapFunction<T, K, V> | Function<T, Iterable<Tuple2<K, V>>> | PairRDD<K, V><br>from a<br>flatMapToPair |
| PairFunction<T, K, V>        | Function<T, Tuple2<K, V>>           | PairRDD<K, V><br>from a mapToPair        |

We can modify [Example 3-28](#), where we squared an RDD of numbers, to produce a `JavaDoubleRDD`, as shown in [Example 3-38](#). This gives us access to the additional `DoubleRDD` specific functions like `mean()` and `variance()`.

*Example 3-38. Creating DoubleRDD in Java*

```
JavaDoubleRDD result = rdd.mapToDouble(
    new DoubleFunction<Integer>() {
        public double call(Integer x) {
            return (double) x * x;
        }
    });
System.out.println(result.mean());
```

## Python

The Python API is structured differently than Java and Scala. In Python all of the functions are implemented on the base RDD class but will fail at runtime if the type of data in the RDD is incorrect.

## Persistence (Caching)

As discussed earlier, Spark RDDs are lazily evaluated, and sometimes we may wish to use the same RDD multiple times. If we do this naively, Spark will recompute the RDD and all of its dependencies each time we call an action on the RDD. This can be especially expensive for iterative algorithms, which look at the data many times. Another trivial example would be doing a count and then writing out the same RDD, as shown in [Example 3-39](#).

*Example 3-39. Double execution in Scala*

```
val result = input.map(x => x*x)
println(result.count())
println(result.collect().mkString(", "))
```

To avoid computing an RDD multiple times, we can ask Spark to *persist* the data. When we ask Spark to persist an RDD, the nodes that compute the RDD store their

partitions. If a node that has data persisted on it fails, Spark will recompute the lost partitions of the data when needed. We can also replicate our data on multiple nodes if we want to be able to handle node failure without slowdown.

Spark has many levels of persistence to choose from based on what our goals are, as you can see in [Table 3-6](#). In Scala ([Example 3-40](#)) and Java, the default `persist()` will store the data in the JVM heap as unserialized objects. In Python, we always serialize the data that persist stores, so the default is instead stored in the JVM heap as pickled objects. When we write data out to disk or off-heap storage, that data is also always serialized.

*Table 3-6. Persistence levels from `org.apache.spark.storage.StorageLevel` and `pyspark.StorageLevel`; if desired we can replicate the data on two machines by adding `_2` to the end of the storage level*

| Level               | Space used | CPU time | In memory | On disk | Comments                                                                                               |
|---------------------|------------|----------|-----------|---------|--------------------------------------------------------------------------------------------------------|
| MEMORY_ONLY         | High       | Low      | Y         | N       |                                                                                                        |
| MEMORY_ONLY_SER     | Low        | High     | Y         | N       |                                                                                                        |
| MEMORY_AND_DISK     | High       | Medium   | Some      | Some    | Spills to disk if there is too much data to fit in memory.                                             |
| MEMORY_AND_DISK_SER | Low        | High     | Some      | Some    | Spills to disk if there is too much data to fit in memory. Stores serialized representation in memory. |
| DISK_ONLY           | Low        | High     | N         | Y       |                                                                                                        |



Off-heap caching is experimental and uses [Tachyon](#). If you are interested in off-heap caching with Spark, [take a look at the Running Spark on Tachyon guide](#).

*Example 3-40. `persist()` in Scala*

```
val result = input.map(x => x * x)
result.persist(StorageLevel.DISK_ONLY)
println(result.count())
println(result.collect().mkString(", "))
```

Notice that we called `persist()` on the RDD before the first action. The `persist()` call on its own doesn't force evaluation.

If you attempt to cache too much data to fit in memory, Spark will automatically evict old partitions using a Least Recently Used (LRU) cache policy. For the memory-only storage levels, it will recompute these partitions the next time they are accessed, while for the memory-and-disk ones, it will write them out to disk. In either case, this means that you don't have to worry about your job breaking if you ask Spark to cache too much data. However, caching unnecessary data can lead to eviction of useful data and more recomputation time.

Finally, RDDs come with a method called `unpersist()` that lets you manually remove them from the cache.

## Conclusion

In this chapter, we have covered the RDD execution model and a large number of common operations on RDDs. If you have gotten here, congratulations—you've learned all the core concepts of working in Spark. In the next chapter, we'll cover a special set of operations available on RDDs of key/value pairs, which are the most common way to aggregate or group together data in parallel. After that, we discuss input and output from a variety of data sources, and more advanced topics in working with `SparkContext`.